# Project structure

Which folders and files the repository has, and what each one is for

Yonatan Haber and Yehonatan Tsitseluk

CellEnMon lab, final-year electrical engineering project

Data-driven discovery of rain-induced and wet-antenna attenuation laws in commercial microwave links

2 November 2026

**In short.** The repository has three parts that depend on each other in one direction. `pipeline/` turns the raw OpenMRG files into a clean data release. `research/` reads that release and does the science. `docs/` explains both. The data itself is in `data/`, which is not in git.

## Contents

1 The parts and how they connect 1

2 Top level 2

3 `docs/`: project documents 2

4 `pipeline/`: the data layer 3

5 `research/`: the science 5

6 `data/`: not in git 9

## 1 The parts and how they connect

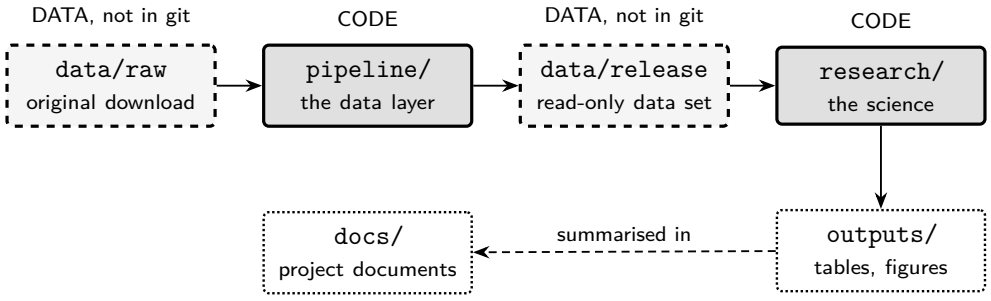



Figure 1: How the parts connect. Thick solid border with grey fill: code. Thick dashed border: data (not in git). Dotted border: documents and results (in git).

The dependency goes one way only: research reads the data through one door (`cmlpipe.api`), and nothing in `pipeline/` uses `research/`. A test checks this.

## 2 Top level

| File | What it is |
|---|---|
| `README.md` | Front page: what the project is and where to start. |
| `CLAUDE.md` | Instructions and rules for Claude (notation, layout, what is protected or frozen). |
| `LICENSE` | MIT licence for the code. |
| `.gitignore` | The only ignore file: keeps the data, the caches and a few big output files out of git. |
| `.gitattributes` | Git settings for line endings and binary files. |
| `environment.yml` | The conda environment to install. |
| `environment.lock.yml` | The same environment with exact versions. |
| `.github/workflows/check.yml` | Runs the fast tests on GitHub on every push. |
| `.claude/commands/updatedocs.md` | The `/updatedocs` command: updates the research log, this document and the reproduce page. |

## 3 `docs/`: project documents

| File | What it is |
|---|---|
| `README.md` | Index of this folder. |
| `proposal.md`, `proposal.docx` | The project proposal (Hebrew); `media/` holds its figure. |
| `ROADMAP.md` | The only roadmap: a checklist, then the detailed WAA plan. |
| `REPRODUCE.md` | How to rerun everything, step by step. Its step table is generated from `research/steps.yaml`. |
| `research_log.tex`, `.pdf` | The research report at the level of ideas: what was done, what it found, what comes next. |
| `project_structure.tex`, `.pdf` | This document. |
| `build_research_log.py` | Builds each PDF from its `.tex` with pdflatex: the two above and the WAA overview. |
| `pipeline_log.pdf` | The report on how the data pipeline was built. |
| `sindy_intro_report_2026-03.pdf` | Our earlier report "Introduction to SINDy". |
| `WAA_research_overview.tex`, `.pdf` | The document of Stage 1 (the WAA equation), a two-column paper: problem, method, results, the comparison of all WAA models, discussion. |
| `WAA_research_overview_v1.tex`, `.pdf` | The first version of that document (kept). |
| `WAA_research_overview_figures/` | Its figures, drawn from saved research tables only: `figs_v2.py` for the paper (`v2_*.png`), `figs.py` for the first version. |
| `reference/` | A previous report of ours used as the model for the paper's format and style. |
| `papers/` | The OpenMRG dataset paper. |

# 4 `pipeline/`: the data layer

The pipeline is the Python package `cmlpipe`. It takes the raw data through three levels and then makes a release: **L1** is the raw data in a standard format (one file per day), **L2** adds quality flags, **L3** is the data reduced to coarser time steps, and the **release** is the read-only data set that the research reads.

## 4.1 Files at the top of the folder

| File | What it is |
|---|---|
| `README.md` | Overview of the pipeline and its stages. |
| `pyproject.toml` | The single list of Python dependencies. |
| `get_data.py` | Downloads OpenMRG from Zenodo into `data/raw` and runs the pipeline. |
| `get_caches.py` | Downloads the saved fits (about 9 hours of fitting), so they need not be repeated. |
| `configs/openmrg.yaml` | All pipeline settings. |
| `start_explorer.bat` | Starts the data explorer on Windows. |
| `LICENSE-DATA.md` | The licence of the data (CC BY-SA, from OpenMRG). |

## 4.2 The code: `src/cmlpipe/`

One folder per job. Most folders have a `core.py` (the calculations only) and a `stage.py` (reads files, calls the core, writes files).

| Folder or file | What it does |
|---|---|
| `io/` | Reads the raw files and writes L1. `openmrg.py`: the CML data. `openmrg_reference.py`: gauges, weather and radar. `schema.py`: defines and checks the L1 format. `netcdf.py`: the one place that reads NetCDF. `ingest.py`: the stages. |
| `qc/` | Quality control: flags bad samples without fixing them. `flags.py`: the flag bits. `rules.py`: the rules. `stage.py`: the stages. |
| `geometry/` | Where links, gauges and radar pixels are, and which radar pixels lie along each link. |
| `aggregate/` | Reduces the 10 s data to coarser time steps (L3), for CML, gauges and radar. |
| `events/` | Finds rain events and dry periods from the references only (never from the CML), and makes the frozen train / test split. `link_state.py`: wet, dry or unknown per link every 5 min. |
| `inventory/` | Which events are significant, how complete each link is, a quality status per sub-link. |
| `release/` | Builds the release. `build.py`: the stage. `cmlstores.py`, `zarrstore.py`: write the time series as Zarr. `metadata.py`: the list of every product. `guide.py`: writes the data guide. |
| `api/` | The only door the research uses to read data. `cmldata.py`: the `CMLData` class. `access.py`: one link with everything on one time axis. `explore.py`: find and plot helpers. |
| `viz/` | Figures. `explorer.py`: plots for the explorer. `quicklooks.py`: one image per event. `link_cards.py`: one page per link. `inventory_figures.py`, `run_report.py` (an HTML summary), `style.py`. |
| `cli.py`, `__main__.py` | The `python -m cmlpipe` commands. |
| `config.py` | Loads the settings file and computes its hash. |
| `provenance.py` | Stamps each output with what produced it. |
| `parallel.py` | Runs the per-day work on several processes. |
| `logging_utils.py` | One log file per run. |

## 4.3 The other folders

| Folder | What is in it |
|---|---|
| `apps/explorer/` | The data explorer (a Streamlit page). `app.py`: the interface. `data.py`: loads the data. `smoke.py`: checks it without a browser. |
| `tests/` | One test file per part of the pipeline (`test_qc.py`, `test_events.py`, ...). Most use a tiny synthetic data set built in `conftest.py`. `test_real_openmrg.py` and `test_api.py` need the real data. `test_boundaries.py` checks that the pipeline never uses the research code. `test_research_steps.py` checks that the step list and the reproduce page match the scripts. |
| `tools/` | `check_boundaries.py`: the checker behind that boundary rule. |
| `examples/` | `quickstart.ipynb`: a short notebook that shows how to read the data. |
| `docs/` | `DATA_GUIDE.md` and `data_dictionary.md`: written by the release stage, not edited by hand. `howto_access.md`: reading data from code. `methods.md`: what each stage does and why. `openmrg_summary.md`: summary of the dataset. `reference/`: a CML libraries how-to and the OpenSense example notebook. |

# 5 `research/`: the science

## 5.1 Shared files

| File or folder | What it is |
|---|---|
| `README.md` | Overview of the research code. |
| `config.yaml` | Every research setting, one section per module. |
| `steps.yaml` | Every step in run order: what it does, what it needs, how long it takes. |
| `__main__.py` | The `python -m research` menu, with `run`, `compare` and `docs`. |
| `common/` | Small helpers used everywhere. `save.py`: save an output. `report.py`: tables for the reports. `plotting.py`: a figure style that does not rely on colour. `config.py`: reads the settings. |

## 5.2 What every module has, and the local folders

Each module below has the same shape: `README.md` (what, status, how to run), `METHOD.md` (the method and the decisions, with numbers), `paths.py` (the only place with file locations) and an `outputs/` folder. Everything in `outputs/` is in git, and every file name starts with the name of the script that made it.

| Folder | What is in it |
|---|---|
| `outputs/products/` | Data that later steps read, for example `baselines.csv`, `tails_index.csv` and the simulated tails. |
| `outputs/tables/` | Results as CSV tables. |
| `outputs/figures/` | Figures (PNG). |
| `outputs/reports/` | One Markdown report per script. |
| `explorations/` | The playground. Local only; just its `README.md` is in git. |
| `.cache/` | Saved fits and logs per script. Local only. It holds the 9 hours of fitting: do not delete. |
| `.vibe-wise/` | Plugin notes. Local only: never commit, move or delete. |

## 5.3 `baseline/`: computing $A(t)$

$A(t) = \mathrm{TSL}(t) - \mathrm{RSL}(t) - \text{baseline}$. This module estimates the baseline.

| File | What it does |
|---|---|
| `baseline.py` | The baseline method itself (functions only), and the baseline table for any event split (used by S8 for the test events). |
| `run_sample.py` | Runs the method on a sample of events and sub-links: v1 (two windows, kept for comparison) or v2 (pre-event window, the current method). |
| `make_dew_products.py` | The hourly dew risk and the list of dry-weather jumps. |
| `make_baseline_table.py` | The frozen baseline v2: one baseline per event and sub-link, with its uncertainty. |
| *In the folder* `validation/`*: the checks of the baseline* | |
| `c1_fake_events.py` | Fake events in dry weather, where the truth is $A = 0$. |
| `c2_after_window.py` | Does the window after the rain carry a drying signal? |
| `c3_daily_cycle.py` | The daily cycle in dry weather. |
| `c4_jumps.py` | What the dry-day jumps are. |
| `c5_flagged.py` | The flagged cases of the v1 run. |
| `c6_sensitivity.py` | Sensitivity to the window length and to mean or median. |
| `r1b_event_level.py` | The final pass or fail of baseline v2. |
| `common.py`, `run_all.py` | Shared code of the checks, and the script that runs them all in order. |

## 5.4 `wet_antenna/`: the wet-antenna attenuation

Two series of steps: T0–T18, the drying-tail exploration (closed; the old step number is in brackets), and S1–S9, the system identification on whole events that replaced it. Besides `README.md` and `METHOD.md` the module has `FINAL.md`, one page that points to the final equation and the WAA document, and `tests/`, the unit test of the equation.

Table 1: The scripts that are run, in run order: the drying-tail exploration.

| **Step** | **Script** | **What it does** |
|---|---|---|
| T0 (0) | `inventory.py` | What data there is for WAA; no analysis. |
| T1 (1) | `extract_tails.py` | Cuts out every drying tail after every rain event, with no selection. |
| T2 (2) | `select_tails.py` | Selects usable tails by rules fixed in advance; tests the "fast drop". |
| T3 (2b) | `refine_selection.py` | Fixes after looking at the selected tails. |
| T4 (2c) | `freeze_tail_set.py` | Last fixes, then freezes tail set v1 that all tail modelling uses. |
| T5 (3a) | `calibrate_simulator.py` | A simulator of tails with known drying laws and realistic noise. |
| T6 (3b-1) | `check_per_tail_fit.py` | Fit one tail at a time on simulated tails (cannot find the true law). |
| T7 (3b-1b) | `check_pooled_fit.py` | Fit all tails together. |
| T8 (3b-1c) | `stress_pooled_fit.py` | Stress test of the pooled fit. |
| T9 (3b-2) | `check_sindy.py` | Weak-form SINDy directly on simulated tails (failed). |
| T10 (3b-3) | `check_free_curve.py` | Learn a free drying curve $k(W)$ from all tails, then name its equation. |
| T11 (3b-3b) | `confirm_free_curve.py` | Confirms that on fresh simulated tails, with the rule written first. |
| T12 (3b-3c) | `check_discovery_method.py` | The rest of the discovery method (model choice, bootstrap band). |
| T13 (3b-4) | `check_quantization.py` | Is the 0.3 dB quantization of RSL the cause of the false step? (no) |
| T14 (4) | `fit_real_drying.py` | The first fit on the real tails, train events only. |
| T15 (4b) | `diagnose_stalled_tails.py` | Why do some real tails stall or rise? |
| T16 (4c) | `characterise_tails.py` | What is in a drying tail, before any law is fitted. |
| T17 (4c2) | `check_drying_after_rain.py` | Does drying start when the rain nearby ends? Data only. |
| T18 (7a) | `short_links.py` | Very short links: enough data, and does the fast first phase grow with length? |

Table 2: The scripts that are run, in run order: the system identification.

| **Step** | **Script** | **What it does** |
|---|---|---|
| S1 | `system_id_ladder.py` | Rain in, attenuation out, on links below 1 km: the model ladder M0–M4. |
| S2 | `system_id_s2.py` | The same on links up to 3 km, the slow part, and the free rung M5. |
| S2b | `system_id_s2b.py` | Repairs of the S2 model (the path / level trap, a bounded slow store, longer fits). |
| S3 | `system_id_s3.py` | Verification of the frozen S2b fits: model-free length test, validation events, directions, gauge input. |
| S3b | `system_id_s3b.py` | The same models fitted on all link lengths, so that the path term is pinned by $L$. |
| S4.1–S4.2 | `conditions_s4.py` | Why events differ: per-event factors, radar against gauges, the conditions. |
| S4.3 | `conditions_s4c.py` | Is the validation gap the radar? The length test with gauge-rescaled rain; heavy events. |
| S5a | `final_s5a.py` | The model refitted on gauge-corrected rain, a last slow-store candidate (M4c), closed forms of $F(R)$ and $P(R)$. |
| S6 | `tighten_s6.py` | Tighten the equation: data funnel, shared shapes, intervals, fake identifiability, closed forms (physics and PySR). |
| S7 | `validate_s7.py` | Validate the frozen S6 candidate without the test set: $W$ against $L$ and the link metadata, short links, dry-path moments, dew, leftovers, radar against gauges. |
| S6b | `return_s6b.py` | The one return to S6: the after-rain misfit cause by cause, drying-time refinements, the box-A rule, the three boxes, the error budget. |
| S8 | `test_s8.py` | Comparison models, the freeze (hashes, committed), then the test set read once with the pass rules. |
| S8b | `test_s8b.py` | A post-freeze evaluation of a pre-freeze model: the closed form of $F$ on the test set, by a rule written first. |
| S9 | `formulate_s9.py` | Formulate Stage 1: the official equation file, the reference of its unit test, the tables behind the document figures and the S9 report. No refitting. |
| S8c | `literature_s8c.py` | After the freeze: our final equation (not refitted) against two published WAA models (Schleiss 2013, Pastorek 2022), fitted on the training events, on the test set. |
| S8d | `comparison_all_models.py` | After the freeze, nothing refitted: every WAA model of S8 and S8c against the model-free length test, the dry-path WAA after rain, the gauge timing and its link factor against the length. |

Table 3: The libraries: used by the scripts above, not run on their own.

| File | What is in it |
|---|---|
| `simulator.py` | The drying laws and the noise model. |
| `per_tail_fit.py` | The candidate models and the fit of single tails. |
| `pooled_fit.py` | The fit of all tails together. |
| `free_curve.py` | The free curve and the naming of its equation. |
| `sindy.py` | A minimal weak-form SINDy, written with numpy only. |
| `tails.py` | Small helpers for the real drying tails (selection, tail classes, rain traces, a tail from any start). |
| `system_id.py` | The models with rain as input (M0–M5), their fit, the event datasets and the leave-one-event-out judge. |
| `length_test.py` | The model-free length test, and running a fitted model on another dataset. |
| `event_factors.py` | Per-event factors on a frozen fit, their fake data, and the radar / gauge ratio per event. |
| `final_model.py` | The model setup of S5a, a fitted model run on another dataset, the closed forms of $F(R)$ and $P(R)$ and their plug-back; the S6 parametrisations and the frozen S6 models. |
| `validation.py` | The frozen S6 models on the datasets, the dry-path windows, event-bootstrap helpers, the error budget and the S8 comparison models. |
| `wind_drying.py` | The two-store model with a drying time that may follow the wind (S6b). |
| `waa_equation.py` | The final WAA equation: a rain series, the link length, the band and the link amplitude in; $W(t)$ and the path part out. |
| `datasets.py` | The noise tables, the tail sets on disk and the bootstraps. |
| `fit_jobs.py` | The long fitting jobs; they save their progress, so an interrupted run continues. |

# 6 `data/`: not in git

| Folder | What is in it |
|---|---|
| `raw/openmrg/` | The original download: `cml/`, `gauges/`, `radar/` and the Torslanda case study. Never modified. |
| `work/openmrg/` | What the pipeline writes: `L1_standard/`, `L2_qc/`, `L3_products/`, `tables/` (with the frozen `events.csv` and the split files), `reports/`, `logs/`. |
| `release/openmrg_v1/` | The read-only release that the research reads: `tables/`, `timeseries/` (Zarr), `catalog.json`, `DATA_GUIDE.md`, `DATA_DICTIONARY.md`, `verification.json`. |